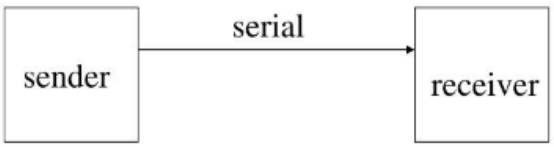
Prezime i ime učenika	:	*Stoiljković Mihajlo*
Odeljenje i grupa	:	*IV-4 grupa 3.*
R. broj i naziv vežbe	:	*Vežba 1.Očitavanje podataka sa senzora koji podržava UART komunikaciju*

IZVEŠTAJ:

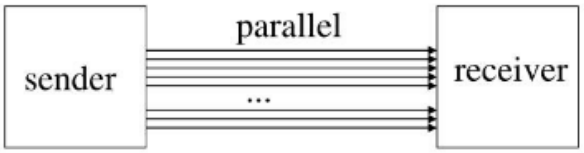
Postoje dva tipa komunikacije i to su:

- Serijska
- Paralelna

Serijska komunikacija je tip komunikacije gde se podaci prenose bit po bit.Brzine prenosa podataka su znatno manje u odnosu na paralelnu komunikaciju.Broj zica koji se koristi je dosta manji (od četiri pa do jedne žice). Pored toga, sama cena je manja zbog manje kompleksnosti izrade i njene jednostavnosti. Na magistrali se često postavlja takt kako bi uređaji mogli da se sihnorizuju odnosno dogovore kada će ko da prima podatak. To predstavlja **serijsku sinhronu magistralu**. Medjutim postoje i serijske magistrale gde se ne prenosi takt već uređaji koji komuniciraju poseduju integrisane uređaje za sinhronizaciju.To predstavlja **serijsku asinhronu magistralu.**



Paralelna komunikacija je tip komunikacije koji vrsi prenos vise digitalnih podataka odnosno bitova istovremeno. Za sam prenos podataka se koristi veca magistrala odnosno imamo vise kanala (žica) i to su najčešće 8, 16 i 32 kanala. Omogucava veoma brz prenos podataka, međutim broj žica koji je potreban je znatno veci u odnosu na serijsku komunikaciju. Zbog svoje kompleksnosti i veceg broja zica ima i vecu cenu. Koristi se često kod delova računara gde se komunikacija ne vrsi bit po bit već su potrebne veće brzine prenosa podataka. Na primer kod **RAM** memorije kao i kod **ISA,ATA,PCI** magistrala i slično.



UART (Universal Asynchronous Receiver-Transmitter) predstavlja asinhronu serijsku magistralu.Postoji i **half duplex, full duplex i simplex** varijanta ove magistrale.Koristi dve žice:

- TX (Transmit line) – linija za prenos podataka
- RX (Receive line) – linija za prijem podataka

Izlaz TX jednog uređaja ide na ulaz RX drugog I obratno.Pored UART komunikacije postoji i USART koji može biti i sihnronizovan.



Primer senzora koji koristi UART komunikaciju je sensor za koncentraciju čestica u vazduhu **SDS011.** Ovaj senzor meri PM2.5 I PM10 čestice u realnom vremenu.

Njegove osnovne karakteristike:



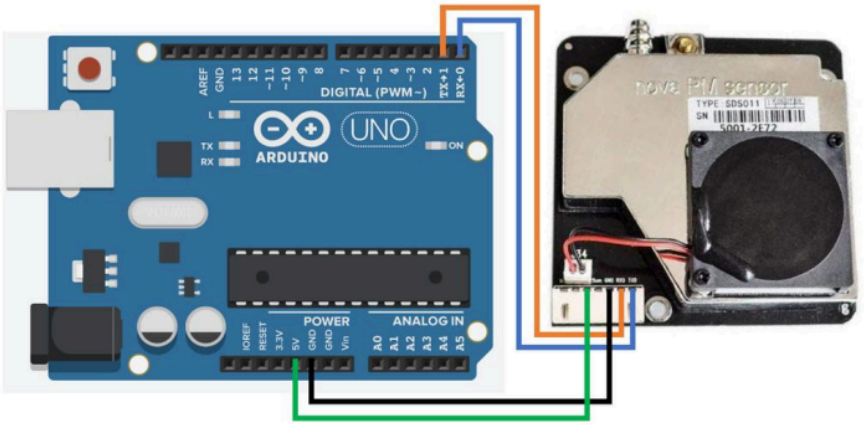
- Proizvođač: **Nova Fitness**
- Merenje: **PM2.5** i **PM10** u realnom vremenu
- Jedinica: **µg/m³**
- Osetljivost: do **0.3 µm** čestica
- Protok vazduha: ima ugrađen ventilator
- Komunikacija: **UART (TX/RX)**
- Napon napajanja: **5V**
- Potrošnja: ~100mA

Čestice:

- **PM2.5** → čestice ≤ 2.5 µm (najopasnije po zdravlje jer prolaze duboko u pluća)
- **PM10** → čestice ≤ 10 µm (prašina, polen, čađ)\

Ovaj senzor radi na principu **laserkog rasejavanja svetlosti.** U njegovoj unutrašnjosti se nalazi mali ventilator koji neprekidno uvlači vazduh u mernu komoru senzora. Kada vazduh uđe, prolazi kroz uski snop laserske svetlosti. U tom trenutku, ako u vazduhu postoje čestice prašine, polena, dima ili drugih zagađivača, laser ih osvetljava i dolazi do **rasipanja svetlosti**.

U senzoru postoji fotodetektor postavljen pod određenim uglom u odnosu na laserski snop. On meri količinu svetlosti koja se odbija i prolazi kroz senzor. Što je više čestica prisutno u vazduhu, to je veće rasipanje svetlosti, a samim tim i jači signal na fotodetektoru. Taj analogni signal se zatim prosleđuje internom mikrokontroleru senzora, koji na osnovu intenziteta i načina rasipanja svetlosti koristi posebne matematičke modele, zasnovane na **Mie-ovoj teoriji raspršenja**, da izračuna broj čestica i njihovu veličinu. Na osnovu toga određuje koncentraciju **PM2.5** i **PM10** čestica, a rezultat se izražava u **mikrogramima po kubnom metru** (µg/m³).



Sa slike vidimo da je potrebno 4 provodnika i usb kabl za vezu arduina sa racunarom.Pinovi koji se koriste za UART komunikaciju su 0 (RX pin) i 1 (TX pin). Kod senzora se koriste pinovi 6 (RX) i 7 (TX).

Za ovaj senzor postoje razne biblioteke koje poseduju različite funkcije u zavisnosti od potrebe,ali za primer ćemo uzeti jednu biblioteku.

```
#include <SdsDustSensor.h>
```

Primer:

```
#include "SdsDustSensor.h"

const int rxPin = 0;
const int txPin = 1;

SdsDustSensor sds(rxPin,txPin);

void setup() {
  Serial.begin(9600);
  sds.begin();
  Serial.println(sds.queryFirmwareVersion().toString()); // ispisuje verziju firmware-a
  Serial.println(sds.setActiveReportingMode().toString()); // postavlja senzor u 'active' reporting modu
  Serial.println(sds.setContinuousWorkingPeriod().toString());
}

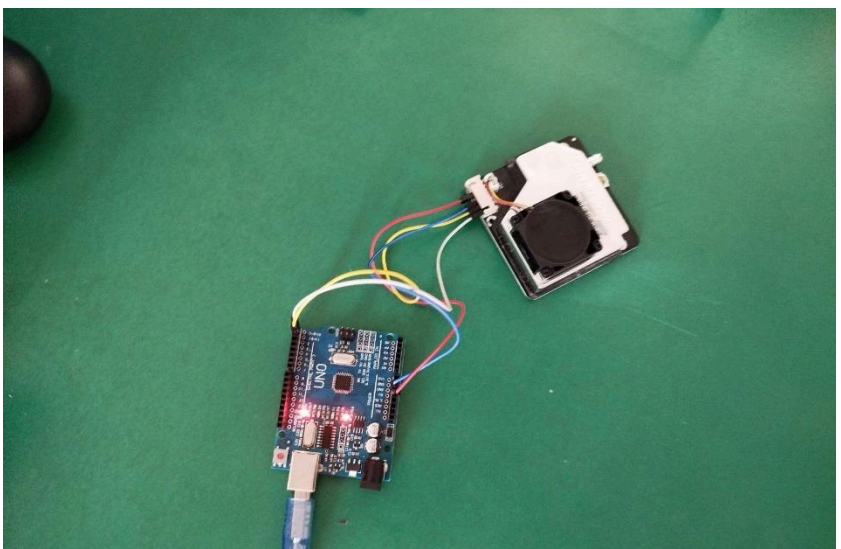
void loop() {
  PmResult pm = sds.readPm();
  if (pm.isOk()) {
    Serial.print("PM2.5 = ");
    Serial.print(pm.pm25);
    Serial.print(", PM10 = ");
    Serial.println(pm.pm10);
    //ako zelite samo da ispisujete merene vrednosti, mozete koristiti toString() metod
    Serial.println(pm.toString());
  } else {
    //loop kasnjenje je podeseno na 0.5s
    Serial.print("Ne mogu da procitam vrednosti iz senzora. Razlog: ");
    Serial.println(pm.statusToString());
  }
  delay(500);
}
```

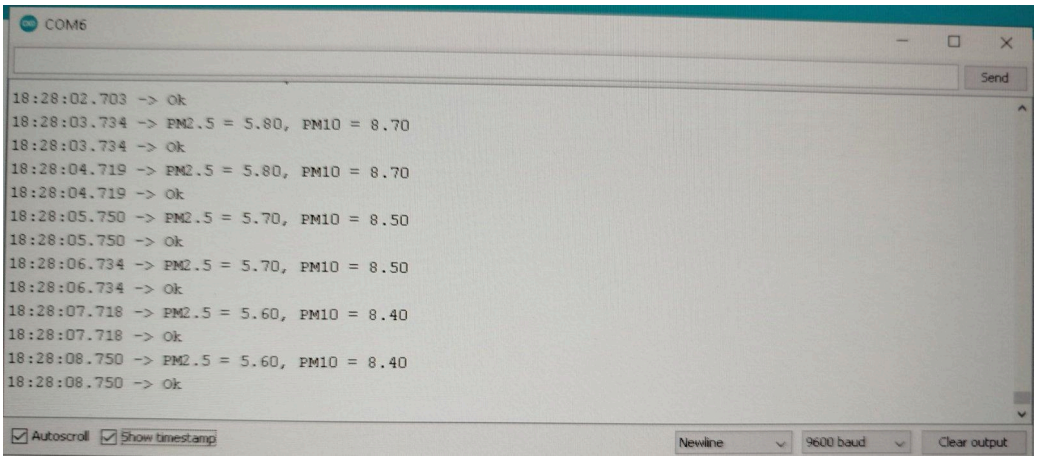
OPIS PROGRAMA:

U ovom primeru koristimo dve konstante koje predstavljaju RX i TX pin na Arduino jer sam senzor koristi UART komunikaciju pa su potrebna ta dva pina. Sds predstavlja objekat klase SdsDustSensor i pomocu sds objekata mi pozivamo razne metode.

U funkciji **setup()**, inicijalizujemo serijski monitor sa 9600 baud rate kao i senzor.Pored toga se koriste metode koji ispisuju informacije od firmware-a i postavljanje senzora u active mode, i podesavanje senzora da radi kontinuirano.Uz svaku tu metodu imamo toString() metodu koja pretvara odredjenu povratnu vrednost metode u string radi ispisa u serijskom monitoru.

U glavnoj petlji odnosno funkciji **loop()** citamo podatke sa senzora preko objekta pm klase PmResult.U daljem izvrsavanju programa imamo uslov da li je citanje uspesno, i ako je citanje uspesno u serijskom monitoru ispisuje vrednosti pm2.5 I pm10 čestica preko objekta pm. Ukoliko čitanje sa senzora nije uspesno, ispisuje poruku za to. Posle svakog izvršavanje loop funkcije pravimo 500ms odziva izmedju očitavanja radi dobijanja real-time podataka u serijskom monitoru.




COM6
Send
18:28:02.703 -> Ok
18:28:03.734 -> PM2.5 = 5.80, PM10 = 8.70
18:28:03.734 -> Ok
18:28:04.719 -> PM2.5 = 5.80, PM10 = 8.70
18:28:04.719 -> Ok
18:28:05.750 -> PM2.5 = 5.70, PM10 = 8.50
18:28:05.750 -> Ok
18:28:06.734 -> PM2.5 = 5.70, PM10 = 8.50
18:28:06.734 -> Ok
18:28:07.718 -> PM2.5 = 5.60, PM10 = 8.40
18:28:07.718 -> Ok
18:28:08.750 -> PM2.5 = 5.60, PM10 = 8.40
18:28:08.750 -> Ok
Autoscroll
Show timestamp
Newline
9600 baud
Clear output